

Dokumentacja Implementacyjna

Projekt: Wizualizacja Grafów Planarnych (JIMP2)

Moduł Obliczeniowy C

Zespół Projektowy:

Bartłomiej Salamon

Piotr Zieliński

Marzec 2026

Spis treści

1	Wstęp	4
1.1	Cel projektu	4
1.2	Zakres funkcjonalności	4
1.3	Przeznaczenie Dokumentu	4
2	Architektura Fizyczna i Podział na Moduły	4
2.1	Struktura plików źródłowych	5
2.2	Schemat Przepływu Sterowania (Control Flow)	5
3	Struktury Danych i Analiza Pamięci (Memory Layout)	6
3.1	Rozmieszczenie i Wyrównanie (Memory Alignment)	6
3.2	Struktura Krawędzi (Edge)	7
3.3	Główny Kontener (Graph)	7
4	Implementacja Algorytmów Obliczeniowych	7
4.1	Moduł: Algorytm Fruchterman-Reingold	7
4.1.1	Złożoność obliczeniowa	7
4.1.2	Mechanika Repulsji (Odpychania)	8
4.1.3	Mechanika Atrakcji (Przyciągania)	8
4.1.4	Aktualizacja pozycji (Cooling Schedule)	8
4.2	Moduł: Osadzanie Tuttego i Eliminacja Gaussa	9
4.2.1	Model matematyczny	9
4.2.2	Inicjalizacja Brzegowa (Boundary Initialization)	9
4.2.3	Eliminacja Gaussa	10
4.2.4	Szczegóły Numeryczne Eliminacji Gaussa	10
5	Struktura Formatów I/O	10
5.1	Dwuprzebiegowy Parser Tekstowy (Two-Pass Scanner)	10
5.1.1	Motywacja projektowa	11
5.1.2	Przebieg I – analiza strukturalna	11
5.1.3	Przebieg II – właściwe wczytywanie danych	12
5.1.4	Walidacja danych wejściowych	12
5.1.5	Złożoność operacyjna	12
6	Specyfikacja pliku binarnego (format eksportu)	12
6.1	Charakterystyka formatu	12
6.2	Zrzut pamięci (Hex Map) dla rekordu wyjściowego	13
6.3	Zalety i ograniczenia formatu binarnego	13

7	Zapewnienie Jakości (Quality Assurance) i Cykl Testowy	13
7.1	Narzędzia Analizy Dynamicznej (Valgrind)	13
7.2	Katalog Błędów Krytycznych (Exit Codes)	14
8	Instrukcja Wdrożeniowa i Kompilacja (Build Process)	15
8.1	Wymagania środowiskowe	15
8.2	Podstawowa kompilacja	15
8.3	Kompilacja etapowa (opcjonalna)	15
8.4	Przykładowy Makefile	15
8.5	Uruchomienie programu	16
8.6	Typowe błędy kompilacji i uruchomienia	16

1 Wstęp

1.1 Cel projektu

Celem projektu jest stworzenie aplikacji konsolowej w języku C, służącej do wyznaczania estetycznych współrzędnych węzłów grafu planarnego na podstawie listy krawędzi. Program stanowi silnik obliczeniowy ("backend"), który przetwarza strukturę grafu i generuje wynik dla modułu wizualizacyjnego (Java). Aplikacja implementuje dwa fundamentalnie różne podejścia do problemu rysowania grafów:

- **Algorytm Fruchtermana-Reingolda** – podejście oparte na symulacji sił fizycznych.
- **Algorytm Tuttego (Tutte Embedding)** – podejście algebraiczne (barycentryczne), gwarantujące poprawny rysunek dla grafów planarnych 3-spójnych.

1.2 Zakres funkcjonalności

Program umożliwia:

1. Wczytanie definicji grafu z pliku tekstowego oraz walidację danych.
2. Wybór algorytmu rozmieszczania.
3. Parametryzację działania.
4. Zapis wyników w formacie tekstowym lub binarnym.
5. Obsługę sytuacji wyjątkowych z precyzyjnymi kodami błędów.

1.3 Przeznaczenie Dokumentu

Głównym zadaniem niniejszego dokumentu jest przybliżenie technicznych detali implementacji aplikacji w języku C. Skupia się on na opisie architektury oprogramowania, organizacji struktur danych w pamięci operacyjnej oraz analizie złożoności zaimplementowanych algorytmów (Fruchterman-Reingold, Tutte Embedding). Opracowanie dokumentuje również rygorystyczne procedury weryfikacji jakości kodu i zapobiegania wyciekom pamięci (QA).

2 Architektura Fizyczna i Podział na Moduły

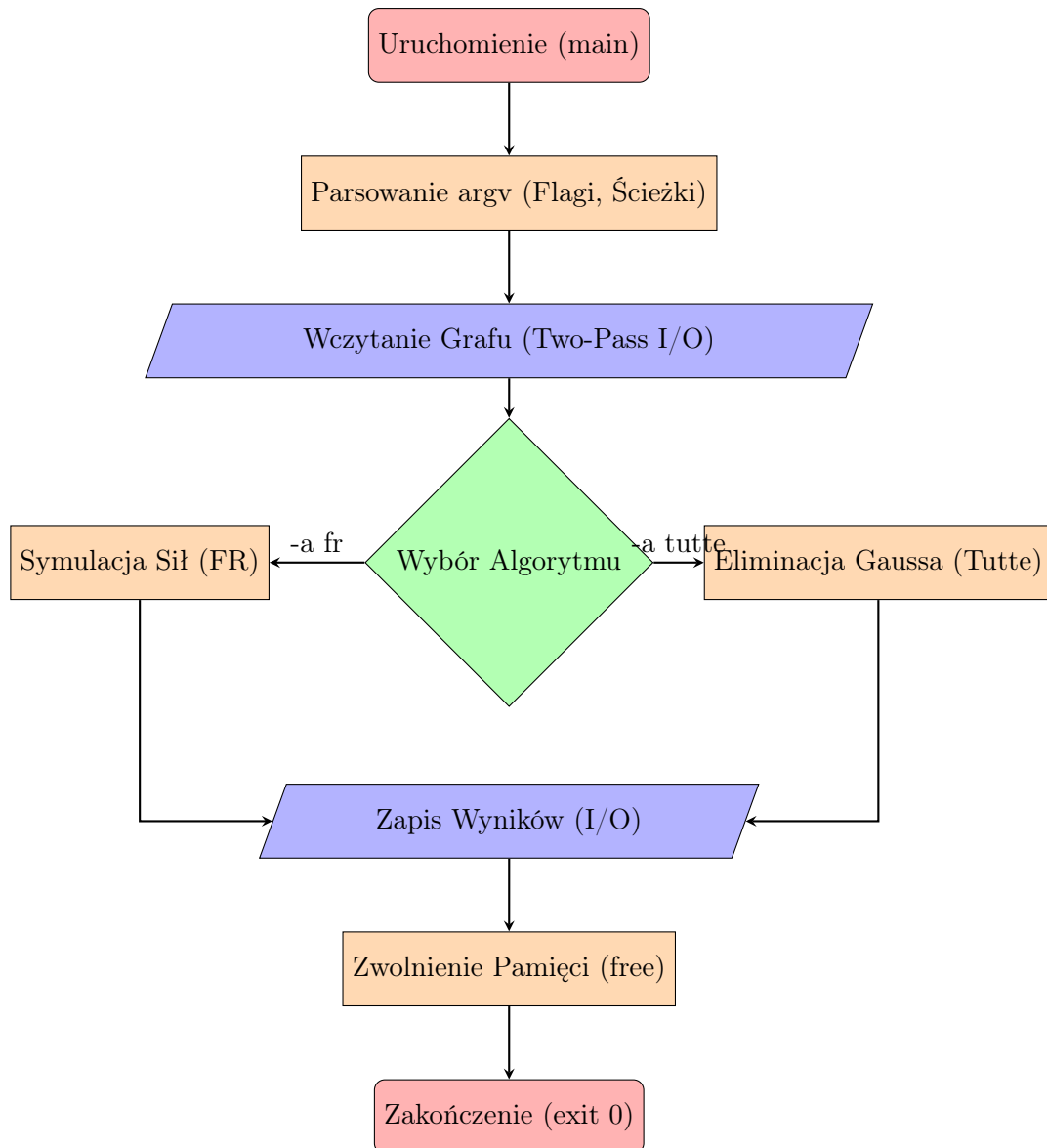
System został napisany w sposób proceduralny i podzielony na moduły. Kod źródłowy rozbito na osobne pliki, dzięki czemu zmiany wymagają ponownej kompilacji tylko części programu, a nie całości. Ułatwia to pracę podczas tworzenia aplikacji oraz pozwala wyraźnie oddzielić logikę programu od operacji wejścia i wyjścia.

2.1 Struktura plików źródłowych

- `main.c` – Punkt wejścia aplikacji. Odpowiada za przetwarzanie argumentów `argv`, obsługę wejścia/wyjścia (odczyt plików tekstowych, zapis wyników) oraz sterowanie wywołaniami modułów obliczeniowych.
- `main.h` – Główny plik nagłówkowy. Zawiera definicje stałych (makra), struktury danych opisujące graf oraz deklaracje funkcji.
- `fruchterman.c` – Zawiera implementację iteracyjnej symulacji układu fizycznego, służącej do wyznaczania rozmieszczenia wierzchołków grafu.
- `tutte.c` – Implementuje podejście algebraiczne do wyznaczania współrzędnych grafu, wykorzystujące metody algebry liniowej (m.in. macierz Laplace’a i eliminację Gaussa).

2.2 Schemat Przepływu Sterowania (Control Flow)

Diagram poniżej przedstawia ilustrację wysokopoziomowego procesu od momentu alokacji do terminacji.



Rysunek 1: Schemat blokowy przepływu sterowania w głównej pętli programu.

3 Struktury Danych i Analiza Pamięci (Memory Layout)

W celu zapewnienia maksymalnej wydajności (optymalizacja chybień pamięci podręcznej - *cache misses*), reprezentacja grafu opiera się na ciągłych wektorach pamięci zamiast list wiązanych. Zrozumienie struktury pamięciowej jest krytyczne dla optymalizacji i debugowania.

3.1 Rozmieszczenie i Wyrównanie (Memory Alignment)

Rozpatrzmy strukturę `Vertex`:

```

1 typedef struct {
2     int id;           // 4 bajty
3     double x, y;      // 2 * 8 bajtów = 16 bajtów
4 } Vertex;
  
```

W architekturze 64-bitowej (x86_64), kompilator C (GCC) dodaje tzw. padding (dopełnienie), aby typ `double` był wyrównany do 8 bajtów. Zatem rozmiar struktury w pamięci wynosi **24 bajty** (4 bajty `int` + 4 bajty paddingu + 16 bajtów `double`), a nie 20 bajtów. Jest to istotne przy szacowaniu maksymalnego rozmiaru grafu mieszczącego się w pamięci L1 Cache procesora.

3.2 Struktura Krawędzi (Edge)

```
1 typedef struct {  
2     int v1, v2;  
3 } Edge;
```

Rozmiar struktury wynosi dokładnie 8 bajtów (brak paddingu). Pola `v1` i `v2` przechowują **indeksy do tablicy vertices**, a nie globalne identyfikatory węzłów. Rozwiązanie to pozwala na ewaluację krawędzi w czasie $O(1)$.

3.3 Główny Kontener (Graph)

```
1 typedef struct {  
2     int vertex_count; // 4 bajty  
3     int edge_count;   // 4 bajty  
4     Vertex *vertices; // 8 bajtów (wskaźnik 64-bit)  
5     Edge *edges;      // 8 bajtów (wskaźnik 64-bit)  
6 } Graph;
```

4 Implementacja Algorytmów Obliczeniowych

4.1 Moduł: Algorytm Fruchterman-Reingold

Plik `fruchterman.c` implementuje heurystyczny algorytm rozmieszczania wierzchołków grafu oparty na modelu fizycznym. W modelu tym:

- wierzchołki traktowane są jako cząstki odpychające się wzajemnie,
- krawędzie działają jak sprężyny przyciągające powiązane wierzchołki.

Algorytm iteracyjnie aktualizuje pozycje wierzchołków, dążąc do osiągnięcia stanu równowagi układu.

4.1.1 Złożoność obliczeniowa

Dominującym elementem kosztu obliczeniowego jest obliczanie sił odpychających pomiędzy wszystkimi parami wierzchołków:

$$O(|V|^2)$$

co czyni algorytm kosztownym dla dużych grafów.

4.1.2 Mechanika Repulsji (Odpychania)

Siła odpychająca pomiędzy dwoma wierzchołkami jest odwrotnie proporcjonalna do ich odległości. Implementacja wykorzystuje odległość euklidesową oraz parametr skalujący k , zależny od liczby wierzchołków.

```
1 double dist = sqrt(x*x + y*y) + 0.01; // zabezpieczenie przed dzieleniem przez 0
2 double force = (k*k) / dist;
3 dx += (x/dist) * force;
4 dy += (y/dist) * force;
```

Uwagi implementacyjne:

- dodanie stałej 0.01 zapobiega niestabilności numerycznej (dzielenie przez zero),
- normalizacja wektora $(x/dist, y/dist)$ zapewnia poprawny kierunek siły,
- wartości sił są akumulowane w zmiennych pomocniczych dx , dy .

4.1.3 Mechanika Atrakcji (Przyciągania)

Oprócz sił odpychających, w układzie działają siły przyciągające, które zachowują się jak sprężyny łączące sąsiadujące wierzchołki. Siła ta rośnie nieliniowo wraz z odległością między powiązаныmi węzłami, zapobiegając nadmiernemu rozproszeniu grafu.

Model matematyczny siły przyciągającej f_a dla odległości d określa wzór:

$$f_a(d) = \frac{d^2}{k}$$

W przeciwieństwie do sił odpychania (które ze względu na charakter elektrostatyczny są ewaluowane dla wszystkich par wierzchołków w czasie $O(V^2)$), siły przyciągania obliczane są wyłącznie dla wierzchołków połączonych krawędzią. Wypadkowy wektor przesunięcia wierzchołka w danej iteracji jest sumą algebraiczną wektorów sił odpychania i przyciągania, co w ostateczności prowadzi system do energetycznego stanu równowagi.

4.1.4 Aktualizacja pozycji (Cooling Schedule)

Po obliczeniu wszystkich sił, pozycje wierzchołków są aktualizowane w oparciu o wypadkowy wektor przesunięcia. Maksymalna długość kroku w pojedynczej iteracji jest rygorystycznie ograniczana przez zmienną temperaturę układu (`temp`), co realizuje założenia symulowanego wyžarzania (ang. *simulated annealing*).

Mechanika chłodzenia:

- **Ograniczenie przesunięcia:** Przesunięcie wierzchołka w osiach X i Y nie może przekroczyć aktualnej wartości temperatury. Zapobiega to gwałtownym oscylacjom i "rozstrzeleniu" grafu w początkowych fazach symulacji, gdy siły odpychania są największe.

- **Dynamiczny spadek temperatury:** Po każdej iteracji temperatura układu jest redukowana o stały współczynnik, odwrotnie proporcjonalny do całkowitej liczby iteracji K . Wraz z upływem czasu układ "stygnie", pozwalając wierzchołkom na coraz drobniejsze, precyzyjne osadzenie się w lokalnych minimach energetycznych.

4.2 Moduł: Osadzanie Tuttego i Eliminacja Gaussa

Plik `tutte.c` implementuje deterministyczny algorytm oparty na algebrze liniowej. W przeciwieństwie do podejścia Fruchterman-Reingold:

- wynik jest jednoznaczny (brak losowości),
- dla grafów planarnych 3-spójnych gwarantowana jest brak przecięć krawędzi,
- problem sprowadza się do rozwiązania układu równań liniowych.

4.2.1 Model matematyczny

Dla każdego wierzchołka wewnętrznego spełnione jest równanie:

$$v_i = \frac{1}{deg(v_i)} \sum_{j \in N(i)} v_j$$

co prowadzi do układu równań:

$$Ax = b$$

rozwiązywanego niezależnie dla współrzędnych x i y .

4.2.2 Inicjalizacja Brzegowa (Boundary Initialization)

Wierzchołki należące do cyklu zewnętrznego są unieruchomione i rozmieszczone na okręgu jednostkowym:

```

1 for(int i=0; i<boundary; i++) {
2     g->vertices[i].x = cos(2*M_PI*i/boundary);
3     g->vertices[i].y = sin(2*M_PI*i/boundary);
4 }
```

Znaczenie:

- zapewnia wypukłość obwiedni grafu,
- stabilizuje rozwiązanie układu równań,
- spełnia warunki twierdzenia Tuttego.

4.2.3 Eliminacja Gaussa

Układ równań liniowych rozwiązywany jest metodą eliminacji Gaussa.

```
1 int pivot = k;
2 for(int i=k+1; i<n; i++) {
3     if(fabs(A[i][k]) > fabs(A[pivot][k])) pivot=i;
4 }
```

Zastosowane techniki numeryczne:

- **Partial Pivoting** – wybór największego elementu w kolumnie jako pivot,
- użycie funkcji `fabs()` w celu porównywania wartości bezwzględnych,
- redukcja błędów zaokrągleń w obliczeniach zmiennoprzecinkowych.

Korzyści:

- zwiększona stabilność numeryczna,
- ograniczenie zjawiska *catastrophic cancellation*,
- poprawna praca dla macierzy rzadkich i źle uwarunkowanych.

4.2.4 Szczegóły Numeryczne Eliminacji Gaussa

Zaimplementowana metoda `gaussian_elimination` rozwiązuje równanie macierzowe $Ax = b$. Implementacja wykorzystuje **częściowy wybór elementu podstawowego (Partial Pivoting)**.

```
1 int pivot = k;
2 for(int i=k+1; i<n; i++) {
3     if(fabs(A[i][k]) > fabs(A[pivot][k])) pivot=i;
4 }
```

Zastosowanie `fabs()` (wartość bezwzględna) oraz pivotowania drastycznie redukuje propagację błędów zmiennoprzecinkowych dla macierzy rzadkich, zapobiegając utracie cyfr znaczących (tzw. zjawisko *catastrophic cancellation*).

5 Struktura Formatów I/O

5.1 Dwuprzebiegowy Parser Tekstowy (Two-Pass Scanner)

Pliki wejściowe są przetwarzane w funkcji `read_graph`. Ze względu na brak nagłówka definiującego rozmiar danych (liczbę wierzchołków i krawędzi), zastosowano strategię dwuprzebiegowego parsowania (ang. *two-pass parsing*), która umożliwia precyzyjną alokację pamięci.

5.1.1 Motywacja projektowa

W języku C dynamiczna alokacja pamięci wymaga znajomości rozmiaru struktur przed ich utworzeniem. Ponieważ plik wejściowy nie zawiera tej informacji:

- niemożliwe jest jednorazowe wczytanie bez kosztownych realokacji (`realloc`),
- dwuprzebiegowy parser eliminuje fragmentację pamięci i zwiększa wydajność.

5.1.2 Przebieg I – analiza strukturalna

Pierwszy przebieg ma charakter analityczny i nie zapisuje danych:

- zliczana jest liczba krawędzi (liczba poprawnych par liczb),
- identyfikowany jest maksymalny identyfikator wierzchołka (`max_id`),
- ignorowane są nadmiarowe tokeny przy użyciu specyfikatorów pomijających.

Zarządzanie nadmiarowymi danymi (Wagi i Nazwy):

Zgodnie z przyjętym kontraktem interfejsów, silnik obliczeniowy C do wyznaczenia pozycji potrzebuje wyłącznie informacji o topologii (połączeniach węzłów). Aby zachować optymalne zużycie pamięci, parser bezpiecznie ignoruje redundantne dane wejściowe, takie jak nazwa krawędzi (ciąg znaków) oraz jej waga.

Wykorzystano do tego specyfikatory formatu pomijającego przypisanie (np. `.*s`, `.*f` w wywołaniach `fscanf`). Pozwala to na poprawne "pochłonięcie" tych tokenów ze strumienia wejściowego bez konieczności alokowania dla nich pamięci w strukturach programu. Dzięki temu główny kontener `Graph` pozostaje lekki, a parser jest w pełni odporny na zróżnicowane długości ciągów tekstowych reprezentujących nazwy krawędzi.

Przykładowy schemat:

```
1 int v1, v2;
2 // .*s pomija nazwe krawedzi, .*f pomija wage
3 while(fscanf(file, ".*s%d.*f", &v1, &v2) == 2) {
4     edge_count++;
5     if(v1 > max_id) max_id = v1;
6     if(v2 > max_id) max_id = v2;
7 }
```

Na tej podstawie ustalane są parametry alokacji:

- liczba wierzchołków: `vertex_count = max_id + 1`
- liczba krawędzi: `edge_count`

5.1.3 Przebieg II – właściwe wczytywanie danych

Po alokacji pamięci:

- wskaźnik pliku zostaje zresetowany (`rewind(file)`),
- dane są ponownie odczytywane i zapisywane do struktur `Graph`.

```
1 rewind(file);
2 int i = 0;
3 while(fscanf(file, "%d_%d", &v1, &v2) == 2) {
4     g->edges[i].v1 = v1;
5     g->edges[i].v2 = v2;
6     i++;
7 }
```

5.1.4 Walidacja danych wejściowych

Parser implementuje podstawowe zabezpieczenia:

- sprawdzanie wartości zwracanej przez `fscanf`,
- ignorowanie niepoprawnych linii,
- obsługa pustych plików,
- kontrola zakresu indeksów (zapobieganie overflow).

5.1.5 Złożoność operacyjna

- czasowa: $O(E)$ dla każdego przebiegu (łącznie $O(2E)$),
- pamięciowa: $O(V + E)$,

co jest optymalne dla reprezentacji grafu w postaci listy krawędzi.

6 Specyfikacja pliku binarnego (format eksportu)

W przypadku użycia flagi `-f bin` dane wyjściowe zapisywane są w formacie binarnym jako sekwencyjny strumień bajtów (ang. *binary dump*). Przyjęty format został zaprojektowany z myślą o wysokiej wydajności operacji zapisu i odczytu, minimalizacji rozmiaru pliku wynikowego oraz bezpośredniej integracji z aplikacją napisaną w języku Java.

6.1 Charakterystyka formatu

- brak separatorów tekstowych (np. spacji, znaków nowej linii),
- zapis w formacie Little-Endian (architektura x86),
- wykorzystanie standardu IEEE 754 dla liczb zmiennoprzecinkowych.

6.2 Zrzut pamięci (Hex Map) dla rekordu wyjściowego

Offset	Rozmiar	Typ danych C	Znaczenie (Java Type)
0x00	4 bajty	uint32_t	Liczba wierzchołków N (int)
<i>Poniższy blok powtarzany N razy</i>			
0x04	4 bajty	uint32_t	ID wierzchołka (int)
0x08	8 bajtów	double (IEEE 754)	Współrzędna X (double)
0x10	8 bajtów	double (IEEE 754)	Współrzędna Y (double)

Tabela 1: Mapa pamięci binowego formatu wyjściowego.

6.3 Zalety i ograniczenia formatu binarnego

Zalety:

- szybki zapis i odczyt,
- mniejszy rozmiar pliku niż format tekstowy,
- brak kosztów parsowania tekstu.

Ograniczenia:

- brak czytelności dla człowieka,
- zależność od architektury (endianness),
- trudniejsze debugowanie bez narzędzi typu hexdump.

7 Zapewnienie Jakości (Quality Assurance) i Cykl Testowy

Proces weryfikacji oprogramowania opiera się na analizie statycznej i dynamicznej, z naciskiem na zarządzanie pamięcią.

7.1 Narzędzia Analizy Dynamicznej (Valgrind)

Aby udowodnić brak wycieków pamięci (Memory Leaks), program poddano testom z wykorzystaniem narzędzia Valgrind:

```
1 valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./graph_layout -g input.txt  
   -o out.txt -a tutte
```

Szczególnej uwadze podlegała implementacja algorytmu Tuttego, gdzie dynamicznie tworzona jest podwójnie wskaźnikowa tablica 2D. W kodzie zrealizowano symetryczną destrukcję macierzy L:

```
1 for(int i=0; i<m; i++) free(L[i]);  
2 free(L); free(bx); free(by); free(rx); free(ry);
```

Wszystkie bloki pamięci (sterowane flagami) są w 100% zwalniane przed kodem wyjścia 0.

Zarządzanie Deskryptorami Plików:

Oprócz monitorowania pamięci sterty (operacje `malloc/free`), rygorystycznej kontroli podlega również zarządzanie zasobami systemu operacyjnego, w szczególności deskryptorami plików. Każde wywołanie otwarcia pliku (`fopen`) posiada zagwarantowane, symetryczne zamknięcie instrukcją `fclose`. Dotyczy to również wszystkich ścieżek obsługi błędów (np. w przypadku natychmiastowego przerwania działania programu z powodu wykrycia nieprawidłowego formatu danych wejściowych w połowie pliku).

Dzięki temu eliminowane jest ryzyko wycieków deskryptorów, które przez narzędzia analizy dynamicznej mogłyby zostać oflagowane jako *still reachable memory*, co gwarantuje pełną stabilność aplikacji nawet przy intensywnym, wielokrotnym uruchamianiu w środowiskach serwerowych.

7.2 Katalog Błędów Krytycznych (Exit Codes)

System reaguje na sytuacje wyjątkowe twardym wyjściem, raportując status do powłoki systemu operacyjnego:

- **Exit 1:** Podana ścieżka do pliku nie istnieje (Input file not found) lub parametry argumentów naruszają specyfikację (np. ujemna liczba iteracji).
- **Exit 2:** Błąd krytyczny I/O – brak możliwości otwarcia pliku (brak uprawnień do odczytu/zapisu).
- **Exit 3:** Nieprawidłowy format danych wejściowych (np. błąd parsowania linii, brak wymaganych wartości jak wagi).
- **Exit 4:** Nieznany algorytm (podano flagę `-a` z nieobsługiwaną nazwą).
- **Exit 5:** Niepoprawny parametr iteracji (Invalid iteration count).
- **Exit 6:** Błąd zapisu danych wyjściowych (np. brak miejsca na dysku).
- **Exit 7:** Nierozpoznany lub niepoprawnie użyty argument (unknown or misplaced argument, błędy składni POSIX).
- **Exit 8:** Błąd alokacji pamięci (`malloc/calloc` zwrócił `NULL`).
- **Exit 9:** Pusty lub niepoprawny graf (brak wierzchołków lub krawędzi).
- **Exit 10:** Graf nie spełnia założeń algorytmu Tuttego (brak planarity lub 3-spójności).
- **Exit 11:** Błąd numeryczny – macierz osobliwa lub niestabilność obliczeń (eliminacja Gaussa).
- **Exit 12:** Przekroczono dopuszczalny rozmiar danych wejściowych (ograniczenia pamięciowe).

8 Instrukcja Wdrożeniowa i Kompilacja (Build Process)

Program został zaimplementowany w standardzie C11 i nie wymaga zewnętrznych bibliotek poza standardową biblioteką matematyczną `<math.h>`. Dzięki temu proces kompilacji jest przenośny i możliwy do wykonania w większości środowisk zgodnych z POSIX.

8.1 Wymagania środowiskowe

- Kompilator zgodny ze standardem C11 (np. GCC) 20
- System operacyjny: Linux / Unix / WSL / macOS
- Dostęp do terminala (powłoka bash)

8.2 Podstawowa kompilacja

```
1 gcc main.c fruchterman.c tutte.c -o graph_layout -lm
```

Opis procesu kompilacji:

- kompilacja wszystkich jednostek translacji (.c) do jednego pliku wykonywalnego,
- automatyczne linkowanie standardowych bibliotek,
- jawne dołączenie biblioteki matematycznej (-lm).

8.3 Kompilacja etapowa (opcjonalna)

Dla większych projektów zaleca się kompilację wieloetapową:

```
1 gcc -c main.c
2 gcc -c fruchterman.c
3 gcc -c tutte.c
4 gcc main.o fruchterman.o tutte.o -o graph_layout -lm
```

Zalety:

- szybsza rekompilacja (tylko zmienione pliki),
- lepsza organizacja projektu,
- możliwość integracji z systemami build (Makefile).

8.4 Przykładowy Makefile

```
1 CC = gcc
2 LDFLAGS = -lm
3
4 all: graph_layout
5
```

```

6 graph_layout: main.o fruchterman.o tutte.o
7     $(CC) main.o fruchterman.o tutte.o -o graph_layout $(LDFLAGS)
8
9 main.o: main.c
10     $(CC) -c main.c
11
12 fruchterman.o: fruchterman.c
13     $(CC) -c fruchterman.c
14
15 tutte.o: tutte.c
16     $(CC) -c tutte.c
17
18 clean:
19     rm -f *.o graph_layout

```

8.5 Uruchomienie programu

Po poprawnej kompilacji, program można uruchomić:

```
1 ./graph_layout [OPCJE]
```

Przykład:

```
1 ./graph_layout -g input.txt -o output.txt -a fr -f bin
```

Dostępne flagi i argumenty:

Flaga	Argument	Opis
-g	<plik>	Ścieżka do pliku wejściowego z grafem (wymagane).
-o	<plik>	Ścieżka do pliku wynikowego (wymagane).
-a	fr tutte	Wybór algorytmu: fr (Fruchterman-Reingold), tutte (Tutte Embedding). Domyślnie: fr .
-f	txt bin	Format wyjściowy: tekstowy lub binarny. Domyślnie: txt .
-i	<liczba>	Liczba iteracji (dotyczy tylko algorytmu siłowego). Domyślnie: 500.
-h	-	Wyświetla pomoc i opis argumentów.

Tabela 2: Opcje konfiguracyjne programu

8.6 Typowe błędy kompilacji i uruchomienia

- **undefined reference to sqrt/sin/cos**

Przyczyna: brak flagi **-lm**

- **undefined reference to main**

Przyczyna: brak funkcji **main()** lub niekompletna kompilacja plików źródłowych

- **segmentation fault (runtime)**

Możliwa przyczyna: błędna alokacja pamięci lub dostęp poza zakresem tablicy

- **multiple definition of**

Przyczyna: wielokrotna definicja tej samej zmiennej lub funkcji w różnych plikach `.c`

- **invalid read / invalid write**

Przyczyna: dostęp do pamięci poza zaalokowanym obszarem

- **use after free**

Przyczyna: użycie pamięci po jej zwolnieniu (`free`)

- **memory leak**

Przyczyna: brak zwolnienia zaalokowanej pamięci

- **implicit declaration of function**

Przyczyna: brak odpowiedniego nagłówka (`#include`)

- **incompatible pointer type**

Przyczyna: niezgodność typów wskaźników

- **assignment makes pointer from integer without a cast**

Przyczyna: przypisanie nieprawidłowej wartości do wskaźnika

- **comparison between signed and unsigned**

Przyczyna: porównywanie typów `signed` i `unsigned`

- **control reaches end of non-void function**

Przyczyna: brak instrukcji `return` w funkcji zwracającej wartość

- **unused variable warning**

Wskazuje na potencjalne błędy logiczne lub nieukończony kod

- **uninitialized variable**

Przyczyna: użycie zmiennej bez wcześniejszej inicjalizacji

- **integer overflow**

Przyczyna: przekroczenie zakresu typu całkowitoliczbowego